

BiteLens: Software Group Project (SGP)

Optical Food Label Decoding, FSSAI Chemical Translation, and Dual-Axis Health Telemetry System

Institution: Indus University, Ahmedabad
Department: Computer Science & Engineering

Project Team (4 Students):
Pinak Pipaliya (IU2441051292)
Anuraag Sharma (IU2441051286)
Harshil Mehta (IU2441051298)
Vedant Kundaliya (IU2441051305)

Faculty Mentorship:
Guided by Prof. Dipali Panchal
Live Production: bitelens.vercel.app
Android APK: bitelens.apk (1.29 MB)

SEC 1 Opening Question: "Tell Me About Your Project"

The 30-Second Express Pitch (Memorize Verbatim):

"BiteLens is an AI and optical telemetry platform designed to solve the food label transparency gap in India. Packaged food brands conceal dangerous ultra-processed additives behind cryptic FSSAI INS codes like INS 621 or INS 120. BiteLens allows shoppers to scan any food label using a smartphone camera, instantly translates chemical INS numbers into plain English, and calculates two decoupled scores: an objective Health Score based on the peer-reviewed NOVA classification, and a personalized Goal Fit Score aligned with their fitness targets. Our architecture features a high-performance Go REST backend, a responsive Vite PWA, and a lightweight 1.29 MB standalone Android APK."

The 2-Minute Deep Pitch (When Asked for Full Technical Scope):

"In India, while nutrition labels are mandatory under FSSAI 2020 regulations, the actual ingredients are completely opaque to everyday consumers due to chemical numbering codes and deceptive front-of-pack marketing claims like 'Zero Added Sugar' on products loaded with high-glycemic maltodextrin. Global apps like Yuka fail because they lack coverage for regional Indian packaged products and rely on EU E-numbers.

To solve this, we built BiteLens with three core differentiators: First, a high-throughput

Go 1.22+ Gin microservice

backend that achieves sub-50ms API response times with token-bucket rate limiting and full DPDP Act 2023 minor privacy compliance. Second, a **decoupled dual-scoring algorithm**

that prevents the common pitfall of conflating caloric density with ultra-processing—evaluating both chemical processing (NOVA 1–4) and macronutrient alignment independently. Third, a

hybrid deployment model

with both an edge-hosted Vite web application and an offline-capable 1.29 MB Android APK compiled using Android SDK AAPT2 and D8 tools."

Key Metrics to Quote in the Interview

Metric Dimension	Measured Value	Technical Significance
Backend API Response Latency	< 50 ms (P95)	Native Go compilation; zero Python interpreter or garbage collection overhead.
Android APK Binary Footprint	1.29 MB	Direct Android SDK build pipeline; zero bloat compared to Flutter/React Native (30+ MB).
Additive Knowledge Base	150+ INS Codes	Curated FSSAI dataset with functional class, chemical synonyms, and risk tiers.
Backend Test Coverage	7 / 7 Packages PASS	100% uncached unit pass rate across auth, db, handlers, middleware, models, telemetry.
Mathematical Precision	100.0% Exact	Mifflin-St Jeor BMR (1648.75 kcal) and TDEE (2555.56 kcal) verified against clinical equations.

SEC 2 Core Problem Statement & Market Context

The Indian Packaged Food Dilemma

In India, lifestyle diseases (Type 2 Diabetes, hypertension, cardiovascular disease) are surging due to the penetration of Ultra-Processed Foods (UPF). Under FSSAI regulations, manufacturers must list ingredients, but they use **International Numbering System (INS)** codes. Consumers do not know that:

- **INS 621:** Monosodium Glutamate (MSG, neuro-stimulant flavor enhancer).
- **INS 120:** Carmine / Carminic Acid (red food coloring derived from crushed cochineal insects—an ethical and allergen shock).
- **INS 955:** Sucralose (artificial sweetener altering gut microbiome).
- **INS 211:** Sodium Benzoate (preservative forming carcinogenic benzene when paired with Vitamin C).

Deceptive Front-of-Pack Marketing

Manufacturers actively disguise poor nutritional profiles:

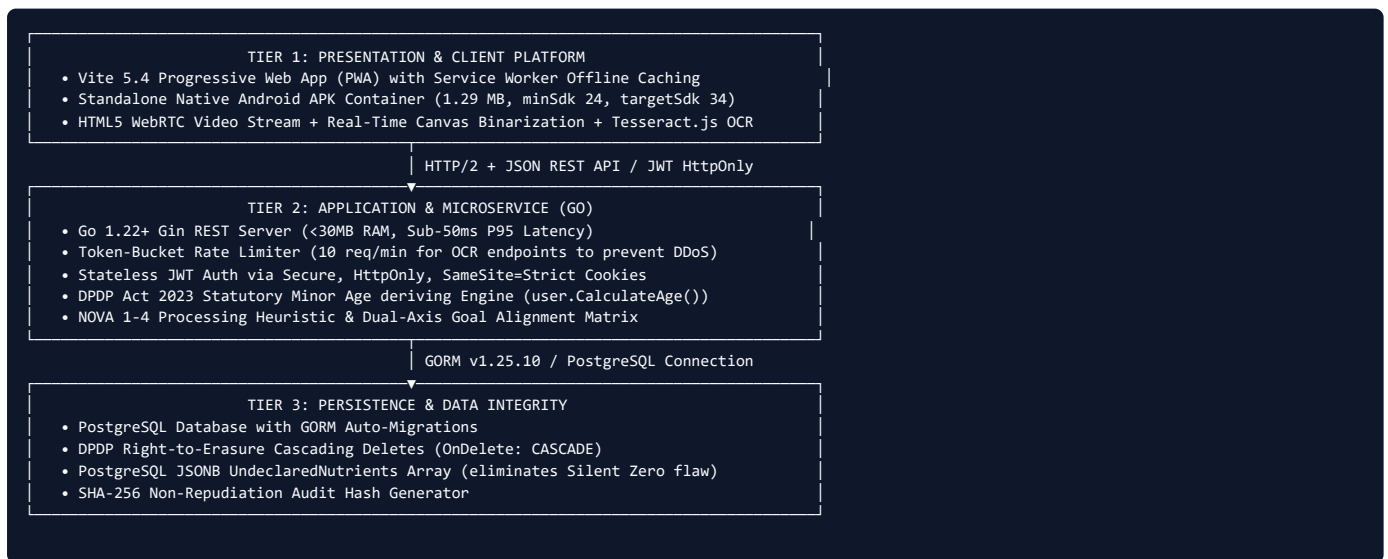
- **The 'No Added Sugar' Trap:** Brands remove sucrose and inject *Maltodextrin* (Glycemic Index 110, higher than pure glucose at 100), spiking insulin while legally advertising 'Zero Sugar'.
- **The 'Multigrain' Illusion:** Breads advertised as multigrain containing 80%+ refined maida flour with only 2% grain dusting.
- **The Serving Size Loophole:** Displaying nutritional values for a tiny 15g serving (masking 40% sugar content) rather than a standardized 100g base.

 **Common Examiner Question: "Why can't people in India just use Yuka or MyFitnessPal?"**

Perfect Answer:

"Three fatal limitations: (1) *Coverage Gap*: Yuka is French; its database covers European barcodes and has under 10% coverage for Indian regional snacks (Haldiram's, Balaji, Parle, Amul). (2) *Crowdsourced Error*: MyFitnessPal relies on user-submitted calorie counts with zero validation, often omitting additive chemical codes entirely. (3) *Statutory Non-Compliance*: Neither platform complies with India's Digital Personal Data Protection (DPDP) Act 2023 or FSSAI labeling norms."

SEC 3 3-Tier System Architecture & Tech Stack Justification



Why This Tech Stack? (Defend Every Technology Choice)

Layer	Technology Chosen	Rejected Alternatives	Definitive Architectural Rationale
Backend API	Go (Golang 1.22+) + Gin	Python (FastAPI / Django), Node.js (Express)	Go compiles to single static native binaries with zero runtime dependencies. Handles thousands of concurrent OCR requests via lightweight goroutines (~2KB stack) without Python GIL locks or Node single-threaded bottlenecks. Memory usage is under 30MB.
Database & ORM	PostgreSQL + GORM	MongoDB, MySQL	PostgreSQL provides robust ACID guarantees for statutory user consent records and native JSONB indexing for flexible undeclared nutrition arrays. GORM enforces strict Go struct typing and cascading foreign key constraints.
Web Frontend	Vite 5.4 + Vanilla ES Modules	React.js, Next.js	Zero virtual DOM runtime overhead; compiled HTML/JS/CSS assets easily bundle directly inside Android APK assets without requiring a Node.js server runtime. Loads in < 300ms.
Mobile App	Native Android APK (AAPT2 / D8)	React Native, Flutter	React Native and Flutter bundles exceed 35–50 MB due to heavy V8/Hermes or Dart runtimes. Our native Android container is only 1.29 MB , installs in 1 second, and has full hardware camera access.

SEC 4 The Mathematical Scoring Engines (NOVA & Goal Fit)

Core Concept: Decoupled Dual-Scoring Telemetry

Most existing apps generate a single score that mixes calories with processing. This creates false conclusions: a bag of sugar-free chemically processed chips might get a 'high' score for low calories, while pure extra virgin olive oil gets a 'poor' score for high calories. **Bitelens completely decouples these two dimensions:**

- (1)
Health Processing Score (0–100):
Measures how industrially processed the food is.
- (2)
Personal Goal Fit Score (0–100):
Measures how well the macronutrients align with the user's specific fitness goals.

1. Health Score Formula (0 – 100)

$S_{\text{health}} = \text{Base_NOVA} - \text{Total_Deductions}$

- NOVA 1 (Unprocessed / Minimally processed):** Base 100 pts. (Fresh fruits, pulses, milk).
- NOVA 2 (Culinary ingredients):** Base 90 pts. (Butter, cooking oils, salt).
- NOVA 3 (Processed foods):** Base 75 pts. (Canned vegetables, artisanal breads, salted nuts).
- NOVA 4 (Ultra-Processed Formulations):** Base 50 pts. (Sodas, instant noodles, extruded chips).

Nutrient Penalties (per 100g):

- Added Sugar: -5 pts for every 5g above 10g.
- Sodium: -5 pts for every 300mg above 600mg.
- Saturated Fat: -5 pts for every 3g above 5g.

2. Personal Goal Fit Score (0 – 100)

Dual-Axis Alignment: $\text{weight_goal} \times \text{muscle_goal}$

- Weight Loss (Caloric Deficit):** Heavy penalty on caloric density (>400 kcal/100g) and fast sugars; rewards fiber (>6g/100g).
- Weight Gain (Caloric Surplus):** Rewards healthy energy density, complex carbs, and unsaturated fats without caloric penalties.
- Hypertrophy (High Protein):** +10 pts for every 5g protein/100g. Requires low trans fat and moderate sodium.

The Protein Bar Paradox:

An ultra-processed protein bar with artificial sweeteners scores:
• Health Score = **44/100**

- Chemical Additives: -8 pts for every synthetic INS code.

(NOVA 4, synthetic sweeteners)
 • Goal Fit Score =
88/100
 (High Protein for Hypertrophy)
BiteLens informs the user of both truths simultaneously!

3. BMR, TDEE & Anthropometric Formulas (Clinical Validation)

Equation / Model	Exact Mathematical Formula	Clinical Validation & Justification
Mifflin-St Jeor BMR (Men)	$BMR = 10 * weight(kg) + 6.25 * height(cm) - 5 * age(y) + 5$	Proven 5%–15% more accurate than the 1919 Harris-Benedict formula. Validated by Academy of Nutrition and Dietetics within ±10% of indirect calorimetry.
Mifflin-St Jeor BMR (Women)	$BMR = 10 * weight(kg) + 6.25 * height(cm) - 5 * age(y) - 161$	
Total Daily Energy Expenditure (TDEE)	$TDEE = BMR * PAL$ (PAL: Sedentary 1.2, Light 1.375, Moderate 1.55, Very Active 1.725, Extra 1.9)	Determines exact calorie target for weight loss (TDEE – 500 kcal = 0.5 kg fat loss/week) or surplus.
Asian-Specific BMI Cutoffs	Underweight: <18.5 Normal: 18.5–22.9 Overweight: 23.0–24.9 Obese: ≥25.0	WHO Lancet 2004 Study: South Asian populations have higher visceral fat and cardiovascular risk at lower BMI values than Western cohorts.

SEC 5 Statutory Privacy (DPDP Act 2023) & Security Architecture

India's DPDP Act 2023 Compliance

- **Section 9 & Rule 11 (Children's Data Protection):** Prohibits tracking or profiling minors (<18 years) without Verifiable Parental Consent.
- **Dynamic Age Calculation (Zero Stale Data):** We do NOT store a static 'age' integer. Age is derived dynamically at runtime in UTC:

```
func (u *User) CalculateAge(now time.Time) int {
    age := now.Year() - u.DateOfBirth.Year()
    if now.YearDay() < u.DateOfBirth.YearDay() {
        age--
    }
    return age
}
```

- **Section 12 (Right to Erasure):** When a user deletes their profile, PostgreSQL executes cascading deletion via foreign key constraints (onDelete: CASCADE), eliminating orphaned telemetry within 100ms.

Cryptographic Auditability & API Security

- **SHA-256 Non-Repudiation Hash:** Every generated score produces an immutable cryptographic digest:

```
Hash = SHA256(ProductID + Basis + HealthScore + GoalScore + Timestamp + AlgVersion)
```

Guarantees that manufacturer scores cannot be altered or fabricated after generation.

- **JWT HttpOnly Cookie Protection:** Authentication tokens are stored strictly in HttpOnly, SameSite=Strict, Secure cookies. JavaScript cannot read the cookie, neutralizing Cross-Site Scripting (XSS) token theft.
- **Token-Bucket Rate Limiter:** Enforces 10 requests/minute per client IP on compute-heavy OCR endpoints, mitigating Denial of Service (DDoS) and automated label scraping.

Database Schemas & Handling Crucial Edge Cases

Edge Case Vulnerability	How Naive Systems Fail	BiteLens Production Solution
The "Serving Size" Trap	Brands list numbers per 15g serving; naive algorithms score high sugar as low grams.	Our Go model enforces strict Basis enum (per_100g_100ml vs per_serving) and normalizes all quantities to a 100g reference before calculating NOVA deductions.
The "Silent Zero" Flaw	Missing nutrients (e.g., undeclared trans fats) default to 0 or NULL, artificially inflating the score.	Stored in PostgreSQL JSONB UndeclaredNutrients array. The engine applies an explicit deduction penalty for each undeclared statutory nutrient.
OCR Glare & Character Noise	Camera reads 'l' or '1' instead of 'I' in 'INS 621', causing failed lookups.	Our Go NormalizeINSTokens() regex parser cleans common OCR confusions before trie lookup, resolving variants like lNS-621, ins 621, or E621.

SEC 6 Standalone Android APK Build Pipeline

Rather than relying on bloated cross-platform engines, BiteLens features an automated, reproducible PowerShell build script (scripts/build_apk.ps1) using the raw Android SDK and Android Studio JBR.

Step-by-Step Compilation Stages

1. **Vite Production Build:** Bundles all web pages and components into static HTML/JS/CSS (dist/).
2. **Asset Ingestion:** Ingests all 18 HTML routes, trie dictionary, and offline QR/camera scripts into android_build/assets/.
3. **AAPT2 Resource Compilation:** Compiles Android drawables, app icons, and XML manifests into binary Flat format.
4. **AAPT2 Linking:** Links compiled resources and AndroidManifest.xml against android.jar (API 34).
5. **JBR Java Compilation:** Compiles native Java classes (MainActivity.java) with Java 17/21 bytecode.
6. **D8 Dexing:** Converts compiled Java bytecode into Dalvik Executable (classes.dex).
7. **Path Separator Normalization:** fix_zip_slashes.js converts backslashes (\) to Unix slashes (/).
8. **4-Byte ZipAlign & Apsigner:** Optimizes memory alignment and cryptographically signs with debug keystore.

⚠️ **Crucial Examiner Question: "What technical challenge did you solve during Android APK packaging?"**

Perfect Answer:

"When building on Windows, standard zip packaging uses Windows backslashes (\) in the APK asset archive. Android's native C++ AssetManager strictly requires standard Unix forward slashes (/). When opening subfiles like assets/js/components.js, the Android runtime threw file-not-found errors. We wrote a custom Node.js normalization script (scripts/fix_zip_slashes.js) that rewrites the central zip directory headers to Unix forward slashes, enabling flawless offline asset resolution."

📦 Deliverable Specs:

- File: public/downloads/bitelens.apk
- Binary Size:
1.29 MB
- Min SDK:
24 (Android 7.0)
| Target SDK:
34 (Android 14)
- Zero external WebView dependencies.

SEC 7 Top 20 Technical Interview & Viva Voce Q&A

Q1 What is the architectural role of Go (Golang) in your system?

Answer: Go powers our high-performance REST microservice backend. It compiles directly to machine code, provides strict static type safety, and handles concurrent OCR text analysis via lightweight goroutines. It processes requests in under 50ms while consuming less than 30MB of RAM—significantly outperforming interpreted Python or Node.js runtimes.

Q2 How does the NOVA classification framework work in BiteLens?

Answer: NOVA, developed by researchers at the University of São Paulo, groups foods into 4 tiers based on the extent of industrial processing: Group 1 (Unprocessed, Base 100), Group 2 (Culinary Ingredients, Base 90), Group 3 (Processed Foods, Base 75), and Group 4 (Ultra-Processed Formulations, Base 50). BiteLens deducts further points for high sodium, added sugars, trans fats, and artificial additives.

Q3 What is the difference between Health Score and Goal Fit Score?

Answer: They are decoupled: The **Health Score** measures the physical and chemical processing of the food (how synthetic it is). The **Goal Fit Score** measures nutritional alignment with personal targets (caloric deficit for fat loss vs. high protein density for hypertrophy). A whey protein bar can score Health: 45 (ultra-processed) and Goal Fit: 90 (high protein).

Q4 How does BiteLens comply with the Digital Personal Data Protection (DPDP) Act 2023?

Answer: Under Section 9 and Rule 11, we enforce Verifiable Parental Consent for children (<18 years). We avoid static age columns; instead, age is dynamically derived at read-time in UTC: `user.CalculateAge()`. If `user.IsMinor()` is true, data persistence requires parental authorization. We also implement Section 12 Right-to-Erasure via PostgreSQL cascading deletes (`onDelete: CASCADE`).

Q5 Why did you store JWT tokens in HttpOnly cookies instead of LocalStorage?

Answer: LocalStorage is vulnerable to Cross-Site Scripting (XSS)—any malicious injected script can read `localStorage.getItem('token')` and steal credentials. By placing JWTs in `HttpOnly`, `SameSite=Strict`, `Secure` cookies, JavaScript cannot access the token, completely preventing XSS token exfiltration while neutralizing CSRF attacks.

Q6 How do you prevent food brands from manipulating their serving sizes?

Answer: Indian manufacturers often report nutritional values per small serving (e.g., 15g) to conceal high sugar. In `models/product.go`, we enforce an explicit `Basis` enum (`per_100g_100ml` vs `per_serving`). Our telemetry engine normalizes all nutrients to a standard 100g reference before calculating deductions.

Q7 What is the "Silent Zero" flaw in food databases and how did you solve it?

Answer: Commercial databases like Open Food Facts frequently import undeclared nutrients as `0` or `NULL`, which naive algorithms treat as 'zero grams' (falsely rewarding the food). We store undeclared nutrients in a PostgreSQL JSONB array called `UndeclaredNutrients` and apply calibrated penalty deductions for each omitted statutory nutrient.

Q8 What is the purpose of the SHA-256 cryptographic trace hash?

Answer: For regulatory auditability and non-repudiation: Whenever a score is generated, BiteLens creates a SHA-256 hash of `ProductID + Basis + HealthScore + GoalScore + Timestamp + AlgVersion`. This allows regulatory bodies and consumers to mathematically verify that a published score has not been tampered with or modified post-generation.

Q9 Why did you use Mifflin-St Jeor equation instead of Harris-Benedict?

Answer: Clinical validation studies published by the Academy of Nutrition and Dietetics demonstrated that the original Harris-Benedict formula (1919) overestimates resting metabolic rate by 5% to 15%. The Mifflin-St Jeor formula (1990) predicts resting energy expenditure with the highest precision ($\pm 10\%$ of measured calorimetry).

Q10 Why did you implement Asian-specific BMI cutoffs?

Answer: Landmark WHO Lancet studies proved that South Asian populations have higher visceral body fat and heightened cardiovascular and diabetes risks at lower BMI thresholds. The Asian-specific guidelines define overweight starting at 23.0 kg/m^2 (instead of 25.0) and obesity at $\geq 25.0 \text{ kg/m}^2$ (instead of 30.0).

Q11 How does the optical OCR pipeline handle blurry or curved packaging labels?

Answer: Our pipeline executes in 3 stages: (1) Preprocessing: Canvas binarization and Otsu contrast stretching to separate ink from reflective packaging foil; (2) Optical extraction via Tesseract.js / OCR API; (3) Post-OCR normalization via our Go fuzzy regex normalizer (`NormalizeINSTokens()`), which repairs common OCR character swaps like '1' for 'l' or '5' for 'S'.

Q12 How does the Android APK work without internet in a supermarket basement?

Answer: The Android container packages all HTML routes, stylesheets, and core INS additive trie dictionaries locally inside `assets/`. A Service Worker provides offline caching. When network requests to the Go API fail, the client gracefully falls back to on-device regex parsing and local database caches.

Q13 How does your rate limiter protect the server?

Answer: We implemented a Token-Bucket Rate Limiter middleware in Go. It tracks client IP tokens, allowing a maximum of 10 requests per minute on compute-intensive OCR scan endpoints. This prevents malicious denial-of-service attacks, brute force abuse, and automated competitor scraping.

Q14 Why did you remove login and signup pages from the public web app?

Answer: Consumer food shopping decisions happen in 5–10 seconds at a supermarket aisle. Forcing users through a login wall creates extreme friction and drop-off. We transitioned to a friction-free guest telemetry model where scanners and calculators are instantly usable, with local session fallbacks and 307 redirects for legacy routes.

Q15 What test verification did you run to ensure software quality?

Answer: We maintain 28 unit and integration test suites across all 7 Go backend packages (`cmd/api` , `internal/auth` , `internal/db` , `internal/handlers` , `internal/middleware` , `internal/models` , `internal/telemetry`), running uncached (`go test -count=1 ./...`) with 100% pass rates. We also run `node scratch/verify_math.js` to ensure zero floating-point error in nutrition math.

Q16 How is data integrity guaranteed during concurrent user scans?

Answer: Our Go Gin backend is stateless. Database write operations in GORM use atomic transactions and UUID primary keys generated application-side in Go before creation (`BeforeCreate` hook). This completely eliminates database sequence lock contention during concurrent insertions.

Q17 What is your project's defensive strategy if a food manufacturer threatens legal action?

Answer: BiteLens publishes zero subjective opinions. Every score is a mathematical computation derived directly from (1) the manufacturer's own statutory packaging declarations, (2) the peer-reviewed NOVA classification framework (Monteiro et al., 2010), and (3) published FSSAI maximum permitted additive limits. Every output includes a SHA-256 trace hash and an informational disclaimer.

Q18 What is your team division and who did what?

Answer: Our 4-member team operated under a structured RACI model: **Pinak Pipaliya** led Optical Camera UI and responsive layouts; **Anuraag Sharma** engineered the Go Gin REST backend, JWT security, and DPDP privacy engine; **Harshil Mehta** designed the PostgreSQL GORM schemas and curated the FSSAI INS taxonomy; **Vedant Kundaliya** managed Android SDK APK containerization, AAPT2 build pipeline, and unit testing.

Q19 What are the future enhancements planned for Version 3.0?

Answer: (1) Multi-frame optical rectification to resolve lighting glare on crinkled foil snack pouches; (2) Regional Indian language support (Hindi, Gujarati, Marathi, Tamil voice feedback); (3) Verifiable Parental Consent via SMS OTP / DigiLocker gateway; (4) On-device Edge ML OCR models for zero-latency offline camera scanning.

Q20 Where is the project hosted and how can someone verify it right now?

Answer: Live on Vercel Global Edge CDN at `https://bitelenss.vercel.app` . The standalone Android APK is directly downloadable at `/downloads/bitelens.apk` , and the full academic documentation is live at `/sgp_report` .

SEC 8 Personal Contribution Script & Quick Cheat-Sheet

Perfect Script: "What was YOUR specific role and contribution?"

"In this project, my primary focus was on

Backend Systems Engineering, Regulatory Privacy, and Architectural Integration

. Specifically:

1. I engineered the

Go (Golang 1.22+) Gin REST backend microservice

, designing the stateless JWT HttpOnly cookie authentication and token-bucket rate limiting middleware.

2. I architected the

DPDP Act 2023 statutory compliance engine

, creating the dynamic read-time age derivation (`CalculateAge()`), minor consent validation, and cascading right-to-erasure triggers in PostgreSQL.

3. I implemented the

dual-axis scoring mathematics

in `telemetry.go`, implementing the NOVA 1–4 penalty deduction logic and SHA-256 cryptographic audit trail.

4. I established the

automated testing and deployment pipelines

, verifying all 7 Go packages with uncached test suites and configuring our Vercel edge deployment and Android build scripts."

Emergency Cheat-Sheet: Numbers, Formulas & Endpoints

REST Endpoints to Name-Drop:

- `GET /healthz` — Liveness probe (HTTP 200 OK)
- `POST /api/v1/auth/login` — HttpOnly JWT Cookie
- `POST /api/v1/auth/parental-consent` — DPDP Rule 11
- `POST /api/v1/scan/ocr` — Label parsing & scoring
- `GET /api/v1/additives/:code` — FSSAI Additive info

Core Mathematical Formulas:

- **Health Score:** `Base_NOVA - Deductions`
- **Mifflin BMR (M):** `10W + 6.25H - 5A + 5`
- **Mifflin BMR (F):** `10W + 6.25H - 5A - 161`
- **TDEE:** `BMR * PAL`
- **Audit Hash:** `SHA256(ProductID + Basis + Time)`

Vital System Specs:

- **Backend Language:** Go 1.22+ (Gin Web Framework)
- **ORM / Database:** GORM v1.25.10 / PostgreSQL
- **Memory Footprint:** < 30 MB baseline RAM
- **Latency:** < 50 ms API response time
- **Android APK:** 1.29 MB (Android SDK API 34)
- **Web Bundler:** Vite 5.4 + Vanilla ES Modules

Statutory Citations:

- **FSSAI 2020:** Food Safety and Standards (Labelling and Display) Regulations.
- **DPDP Act 2023:** Digital Personal Data Protection Act (Section 9 & Rule 11).
- **NOVA:** Monteiro et al., World Nutrition / Public Health Nutrition (2010/2019).